

# Acceleration of Quantum Kernel Methods for Image Classification Tasks

Project Work Parallel Programming

**Fouepe Dylan**

 University of Florence

 Parallel Programming – 2025

# Overview

1. Context & Scope
2. Experimental Methodology
3. HPC Architecture
4. Performance Benchmark
5. Classification Results
6. Failure Analysis: Kernel Collapse
7. Conclusion & Perspectives

## Context & Scope

---

## 💡 Quantum Kernel SVMs

- Project classical data into a  $2^Q$ -dimensional Hilbert space.
- Fidelity kernel:  $k(x_i, x_j) = |\langle \psi(x_i) | \psi(x_j) \rangle|^2$ .
- At  $Q = 16$ : 65,536 complex amplitudes per state.
- Guaranteed convex SVM optimization.

## ⚠️ HPC Bottleneck

- Gram matrix cost:  $\mathcal{O}(N^2 \cdot 2^Q)$ .
- GPU/CUDA acceleration is essential at scale.

## 🎯 Three Research Questions

1. How do classical and quantum-kernel SVMs compare on the same binary tasks?
2. Which backend offers the best throughput-runtime trade-off at 16 qubits?
3. Which empirical signals indicate kernel concentration?

# Experimental Methodology

---

# Datasets & Binary Tasks

Three datasets, binarized into three difficulty levels:

| Dataset       | Easy   | Med    | Hard   |
|---------------|--------|--------|--------|
| Fashion-MNIST | 0 vs 1 | 3 vs 8 | 0 vs 6 |
| CIFAR-10      | 1 vs 6 | 0 vs 2 | 3 vs 5 |
| SVHN          | 1 vs 0 | 2 vs 7 | 3 vs 5 |

## Default Setup

- $Q = 16$  qubits  $\rightarrow 2^{16}$  amplitudes
- **YZ ansatz** with entangling layers (fixed weights)
- 3-fold CV + Optuna for hyperparameter  $C$
- Metrics: F1, AUC, runtime, SV fraction, kernel std

# Preprocessing Pipeline & Fidelity Kernel

## Feature pipeline:

1. Extract pixels and flatten
2. PCA to  $d = 16$  components
3. Range scaling
4. Encode via *angle map*, *Y map* or *YZ map*
5. Simulate states via PennyLane

## Fidelity kernel:

$$K_{ij} = \left| \sum_{k=1}^{2^Q} S_A[i, k] \overline{S_B[j, k]} \right|^2$$

## Training protocol:

- Train/test split from dataset
- 3-fold CV on training partition
- Hyperparameter search (C) via Optuna
- Final re-fit on train+val, eval on test set

### **i** Kernel-based, not variational

Weights are fixed at init – kernel method, not a training loop.

# HPC Architecture

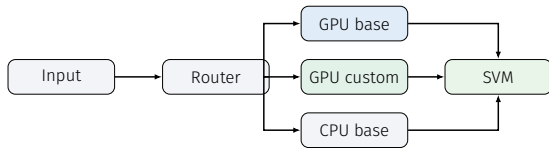
---



# Unified Backend API

Three execution paths, same normalization and SVM:

- **CPU base** – NumPy reference, no VRAM tiling
- **GPU base** – PyTorch GPU matmul + pinned memory + CUDA streams
- **GPU custom** – PyTorch states → DLPack → CuPy → custom CUDA kernel (FP64)



SVM solver always runs on CPU.

# Backend Optimizations & Shared Normalization

## Key optimizations:

- **VRAM-aware tiling:** block size auto-tuned to budget
- **Kernel autotuning:** tile sizes cached persistently
- **Bulk precompute:** states prebuilt when memory allows
- **Stream pool:** CUDA stream orchestration
- Best: vram\_fraction=0.95, num\_streams=1, GPU base pinned+streams

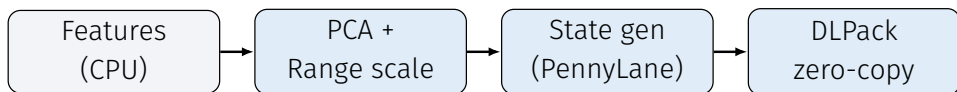
## Shared Normalization

All backends use identical post-processing:

$$\hat{K}_{ij} = \frac{K(X, Y)_{ij}}{\sqrt{K(X, X)_{ii} K(Y, Y)_{jj}}}$$

- NaN/Inf checks before writing
- Diagonal validation before SVM fit
- Guarantees fair comparison

# GPU Custom: Kernel Construction Pipeline



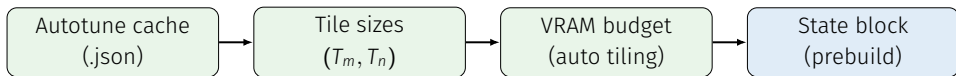
## Host preprocessing (step 1/3):

- Raw image pixels extracted and flattened
- PCA to 16 components + range scaling
- Quantum states generated via PennyLane in **FP64**
- Output: complex state matrix  $S \in \mathbb{C}^{N \times 2^Q}$

### Why FP64?

FP32 leads to **kernel collapse** ( $AUC \approx 0.5$ ) due to floating-point cancellation in the fidelity dot products at  $2^{16}$  dimensions.

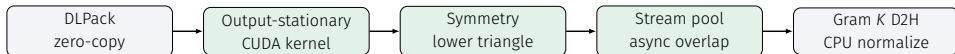
## GPU Custom: Control & Cache Layer (2/3)



How the control layer works:

- **Autotune cache:** tile sizes ( $T_m, T_n$ ) measured once per (nq, dtype) and saved to JSON
- **VRAM-aware tiling:** vram\_fraction=0.95 caps state block size to available GPU memory
- **Bulk precompute:** if the full block fits, states are prebuilt to reduce PennyLane round-trips
- Optimal profile in benchmarks: **1 stream** with **pinned memory** (GPU base path)

## GPU Custom: Device Core (3/3)



### Device execution:

- **DLPack:** PyTorch GPU tensor → CuPy with **zero memory copy**
- **Output-stationary:** each CUDA thread writes exactly one  $K_{ij}$  (no write conflicts)
- **Symmetry:** lower triangle only; then  $K_{ji} \leftarrow K_{ij}$  mirrors upper entries
- **Stream pool:** asynchronous block overlap between state prep and Gram compute
- Final step: D2H transfer of  $K$  and CPU-side cross-normalization

# Performance Benchmark

---

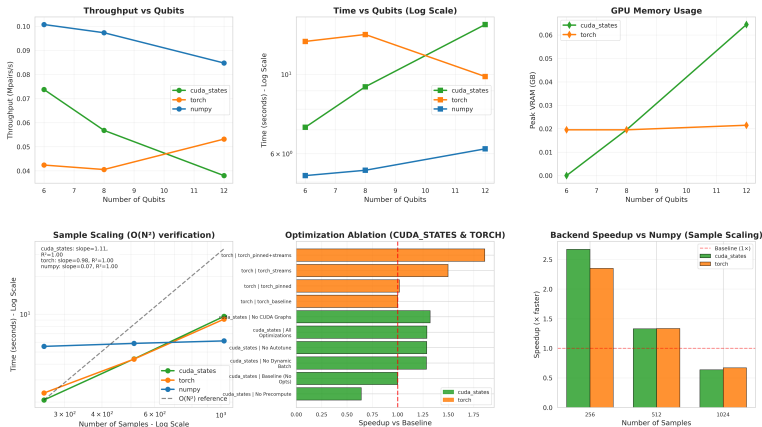
# Throughput by Dataset & Backend (Mpairs/s)

Highest qubit setting per dataset (Fashion: 12 qubits; CIFAR-10/SVHN: 16 qubits)

| Dataset | Size | Qubits | CPU base      | GPU base      | GPU custom    |
|---------|------|--------|---------------|---------------|---------------|
| fashion | 256  | 12     | 0.0059        | 0.0140        | <b>0.0159</b> |
| fashion | 512  | 12     | 0.0224        | <b>0.0299</b> | 0.0298        |
| fashion | 1024 | 12     | <b>0.0856</b> | 0.0575        | 0.0546        |
| cifar10 | 256  | 16     | 0.0020        | 0.0057        | <b>0.0062</b> |
| cifar10 | 512  | 16     | 0.0086        | <b>0.0175</b> | 0.0112        |
| cifar10 | 1024 | 16     | 0.0319        | <b>0.0375</b> | 0.0245        |
| svhn    | 256  | 16     | 0.0023        | 0.0053        | <b>0.0058</b> |
| svhn    | 512  | 16     | 0.0097        | <b>0.0175</b> | 0.0124        |
| svhn    | 1024 | 16     | 0.0328        | <b>0.0366</b> | 0.0243        |

At size 256: GPU backends dominate. At size 512: GPU base fastest. At size 1024: CPU base recovers on Fashion.

# Benchmark Profile – Fashion-MNIST



Backend ranking is **not global**: it varies with sample size and dataset profile.



# Benchmark Profiles – CIFAR-10 & SVHN

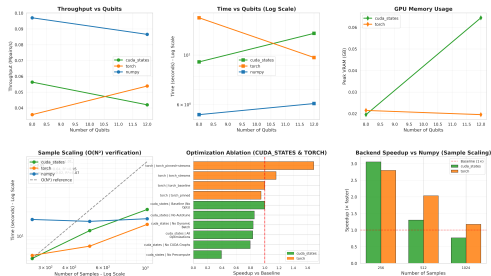


Figure 1: CIFAR-10 (16 qubits)

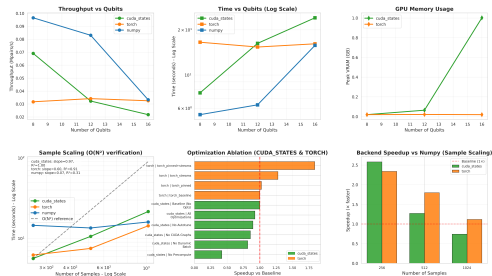


Figure 2: SVHN (16 qubits)

GPU base is fastest across most 16-qubit configurations. Data movement dominates over stream complexity.

## Classification Results

---

# Quantum Backend Quality vs Runtime

Mean F1, AUC and runtime – averaged over difficulty levels, latest run extraction

| Dataset | Size | Backend         | Mean F1       | Mean AUC      | Mean time (s) |
|---------|------|-----------------|---------------|---------------|---------------|
| cifar10 | 500  | GPU custom      | 0.3500        | 0.4480        | 336.13        |
| cifar10 | 500  | GPU base        | 0.4270        | 0.4812        | 190.61        |
| cifar10 | 1000 | GPU custom      | 0.4305        | 0.4365        | 496.73        |
| cifar10 | 1000 | GPU base        | 0.5729        | 0.5532        | 302.19        |
| fashion | 500  | GPU custom      | 0.8905        | 0.9569        | 237.05        |
| fashion | 500  | <b>GPU base</b> | <b>0.8889</b> | <b>0.9574</b> | <b>110.78</b> |
| fashion | 1000 | GPU custom      | 0.8958        | 0.9553        | 339.29        |
| fashion | 1000 | <b>GPU base</b> | <b>0.8998</b> | <b>0.9557</b> | <b>187.13</b> |
| svhn    | 500  | GPU custom      | 0.1789        | 0.4719        | 545.39        |
| svhn    | 500  | GPU base        | 0.1547        | 0.4719        | 302.73        |
| svhn    | 1000 | GPU custom      | 0.1596        | 0.4846        | 637.30        |
| svhn    | 1000 | GPU base        | 0.2998        | 0.4846        | 380.61        |

**GPU base** generally faster and more accurate. Fashion strong; SVHN/CIFAR-10 degrade.

# Classical vs Quantum: Paired Comparison

Positive  $\Delta F1/\Delta AUC$  = classical better; negative  $\Delta Time$  = classical faster

| Comparison             | Size | $\Delta F1$ | $\Delta AUC$ | $\Delta Time$ (s) | Wins C/T/Q |
|------------------------|------|-------------|--------------|-------------------|------------|
| Classical – GPU base   | 500  | +0.1708     | +0.1488      | −188.28           | 8/0/1      |
| Classical – GPU base   | 1000 | +0.1721     | +0.1600      | −274.52           | 8/1/0      |
| Classical – GPU custom | 500  | +0.1879     | +0.1600      | −359.76           | 8/0/1      |
| Classical – GPU custom | 1000 | +0.2676     | +0.1990      | −475.66           | 8/1/0      |

- Classical **dominates**: wins 8/9 paired comparisons at both sample sizes
- Quality gap **widens at 1000 samples** – quantum kernels do not scale better with data
- Acceleration improves throughput but does **not remove representational limits**

# Failure Analysis: Kernel Collapse

---

# Hard Tasks: Quantitative Failure Pattern

## Observable collapse signals:

- Inter-class gap  $\approx 0$  after normalization
- Kernel std  $\in [10^{-4}, 10^{-3}]$
- SV fraction  $\rightarrow 1.0$
- AUC  $\approx 0.5$  (random)

## Hard CIFAR-10 with 6 layers:

- *kernel std* = 0.0005
- *SV frac* = 1.0000
- *test F1* = 0.0000
- *test AUC* = 0.4757

## ⊘ Kernel Concentration

States become nearly **indistinguishable**.  
All samples become support vectors –  
the SVM cannot separate classes.

## Contributing factors:

- PCA/grayscale information loss
- Generic ansatz, not image-aware
- Deeper circuits  $\rightarrow$  more concentration
- Global fidelity kernel limitation

## Difficulty-Level Summary Across Backends

Mean F1, AUC and time per difficulty (averaged over datasets and sizes, latest run)

| Difficulty / Backend | F1     | AUC    | Time (s) |
|----------------------|--------|--------|----------|
| easy / GPU custom    | 0.4907 | 0.6327 | 475.44   |
| easy / GPU base      | 0.6366 | 0.6499 | 258.76   |
| med / GPU custom     | 0.5263 | 0.6314 | 404.06   |
| med / GPU base       | 0.6560 | 0.6896 | 226.39   |
| hard / GPU custom    | 0.4357 | 0.6125 | 416.44   |
| hard / GPU base      | 0.3289 | 0.6126 | 251.88   |

- **GPU base** outperforms at easy/med – faster and more accurate
- At **hard**, both backends collapse: F1 drops, AUC  $\rightarrow$  0.5
- Compute acceleration cannot resolve the representational bottleneck

## Conclusion & Perspectives

---



# Conclusion

- ✔ **HPC Pipeline:** Three backends share a unified API. GPU paths dominate small sizes; ranking varies with dataset and sample count.
- 📈 **Fashion quality:** Both quantum backends competitive (F1 up to **0.9653**, AUC up to **0.9956** for GPU base, size 1000).
- ❗ **Classical dominates:** RBF SVM wins 8/9 paired comparisons at both sample sizes. Gap widens at 1000 samples.
- ⊘ **Kernel collapse on hard tasks:** Concentration signals ( $\text{std} \sim 10^{-4}$ , SV frac = 1). Acceleration alone does not fix this.

## → Better Feature Maps

- Projected/local kernels to reduce concentration
- Geometry-aware ansatz for image structure
- Stronger classical front-end before quantum embedding

## → Richer Diagnostics

- Gap trajectories and kernel spectra
- SV dynamics to detect collapse earlier
- Broader benchmark grids per dataset

## → HPC Scaling

- Multi-GPU / MPI for large Gram builds
- GPU-side SVM solvers to remove CPU bottleneck

# Thank You!

 [github.com/DylanUnifi](https://github.com/DylanUnifi)

 University of Florence

## Questions?